

长时程自主 Agent

从完成任务到持续经营任务

2026.08 | 基于 20+ 行业来源的深度调研

Agent 能跑多久？

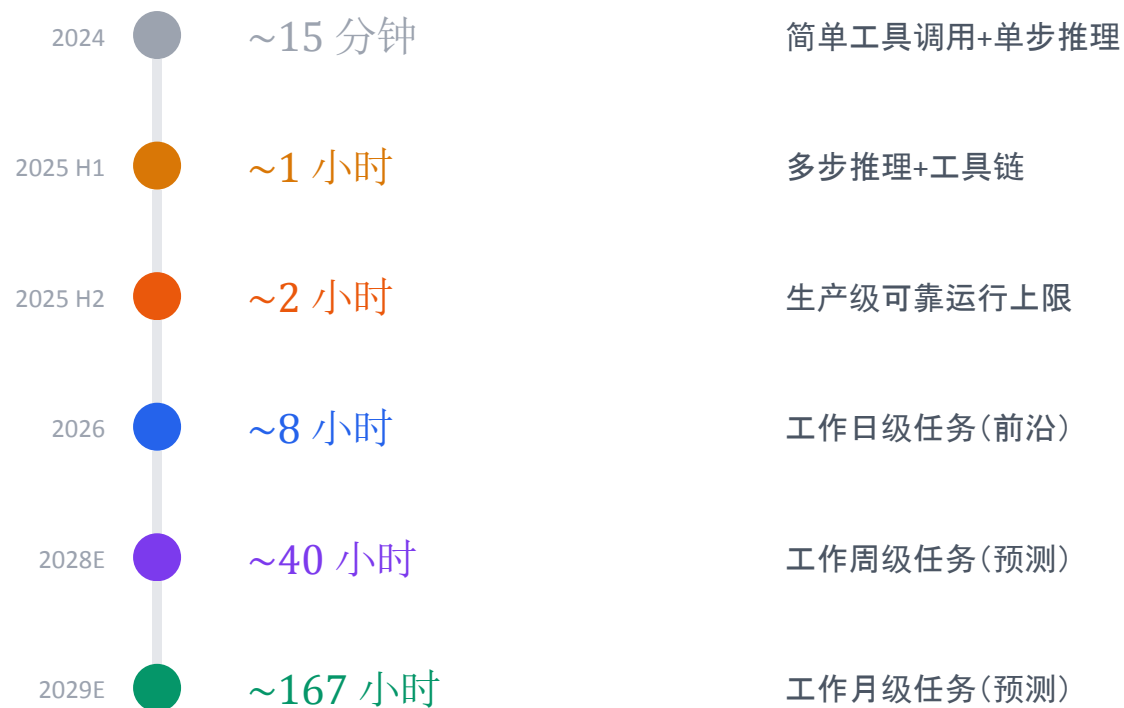
分钟 → 小时 → 天 → 周

2026 年，我们正在见证 Agent 从"一次性回答问题"到"持续经营任务"的范式跃迁。

任务时长每 7 个月翻倍，但失败率随时长二次方增长。这意味着什么？

任务时长每 7 个月翻倍

METR 自 2019 年持续追踪 AI Agent 自主任务时长, 发现了指数级增长趋势:



这些"长时程"到底有几种形态? →

三种长时程形态

长程推理

Long-Horizon Reasoning

模型规划 8+ 小时任务, 分解为子目标链, 50% 可靠性(2026)

典型场景: 复杂代码重构、多文档分析

长时运行

Long-Running Execution

进程跨数千次模型调用持续运行, 跨会话/沙箱维持状态恢复

典型场景: 持续集成监控、数据管道维护

持久身份

Persistent Agency

Agent 拥有持久身份, 跨周/月积累记忆、偏好和用户模型

典型场景: 个人助理、客户关系管理

四个前沿实践

30+ 小时

Anthropic Managed Agents

Brain-Hands-Session 分离架构, 11,000 行代码单次产出

天级任务

Google Agent Runtime

支持 Agent 自主运行数天, 含 Memory Bank + Agent Identity

8h 重构

Cursor Background Agents

云端长时运行, 关盖不中断, 本地/远端无缝切换

1 个月

Project Vend (Anthropic)

Claude 运营自动售货机: 库存/定价/供应商沟通, 暴露周级身份一致性问题

EdgeBench: 长时程 Agent 的标尺

ByteDance Seed 发布, 评测 Agent 在 12-72 小时持续交互中的学习和改进能力

134

个真实世界任务

任务数量

6

科学/工程/知识/优化/数学/游戏

任务类别

38,000+

小时 Agent 交互数据

总交互时长

12

小时(最长 72h+)

每任务最低时长

57.2

小时/任务(最高 320h)

人类专家平均用时

51.3%

Claude Opus 4.8 @12h

领先模型得分

缩放律: Agent 学习遵循 Log-Sigmoid 曲线

$$R^2 = 0.998$$

Agent 在环境中的学习曲线遵循高度精确的
log-sigmoid 数学函数

学习速度每 3 个月翻倍

跨越 2025.9 到 2026.5 的前沿模型代际

三种学习轨迹:

稳步提升

持续线性改进

快速起步

早期大幅提升后趋平

后期突破

长时间平台期后突然跃升

关键洞察: 真正的性能跃升来自问题重构 (problem reformulation), 而非简单的迭代调优

数据很乐观, 但长时运行为什么这么难? →

六大漂移机制

Agent 运行时间越长, "跑偏"的方式越多。以下是六种已被 识别的漂移类型:

目标漂移

Agent 静默偏离原始目标

上下文漂移

过时信息稀释 Transformer 注意力

角色漂移

行为风格与指定人设不一致

工具使用漂移

工具调用模式从多样变为重复

幻觉级联

推理输出逐步脱离事实

计划衰退

多步策略在执行中逐渐退化

其中最致命的是上下文漂移 — 65% 的失败源于此 →

上下文漂移: 65% 失败的根源

65%

企业 AI Agent 生产失败
源于上下文漂移和记忆丢失

— Zylos Research

关键发现

- 退化在上下文窗口 60-70% 处开始
不是在上限处
- 更大的窗口只是延迟发现漂移
不能解决漂移
- 问题根源是架构设计
不是模型参数规模

机制解析

随着 Agent 处理工具输出、API 响应和中间结果, 目标相关信息与噪声的比例持续下降。模型的模式匹配机制开始更多地关注最近的上下文, 而非窗口开头的显式指令。初始系统提示的权重被逐步稀释, 最终导致行为偏移。

[更量化地看: 时长和失败率的关系 →](#)

二次方失败律

任务时长 $\times 2 \rightarrow$ 失败率 $\times 4$

在当前架构下，失败率随任务时长呈二次方增长，而非线性增长

这意味着什么？

2h 任务成功率 80%

→

4h 任务成功率仅 ~36%

扩大上下文窗口

→

延迟漂移发生时间，不能消除漂移

简单重试策略

→

在长时任务上成本爆炸但效果递减

问题清楚了。怎么从架构上解决？→ 七大关键组件 →

长时程 Agent 的七大关键组件

1

Session-as-Event-Log

追加式事件日志, 支持恢复和 审计

2

Brain-Hands-Session 分离

推理/执行/历史解耦, 独立替换

3

外化状态文件

prd.json / progress.txt / AGENTS.md

4

检查点与恢复

中间状态保存, 防止从零重启

5

委托审批机制

人工暂停/恢复, 零计算成本等待

6

生成/评估分离

Planner/Worker/Judge 防自我打分

7

上下文重置与压缩

结构化交接文件, 模拟团队换班

Session-as-Event-Log 架构

每个 thought、tool call、observation 都被追加到不可变事件日志中。

故障恢复

从事件日志重建会话状态，不丢失任何进度

可调试性

完整的执行轨迹，每一步都可回溯审计

性能优化

Anthropic: p50 首 token 延迟降 60%, p95 降 90%+

分离关注点

日志独立于模型上下文窗口，不受窗口大小限制

Brain-Hands-Session 分离架构



关键设计原则

三者可独立替换和扩展。Brain 崩溃不影响 Session 历史，Hands 沙箱销毁不影响推理状态。这种解耦防止了级联故障 — 长时运行的 Agent 最怕的就是一个组件故障导致全部重来。

状态需要外化到文件系统 — 怎么做？ →

外化状态:记在上下文窗口外

"只存在于上下文窗口中的目标, 终将被遗忘、覆盖或稀释"

prd.json

任务清单 — Agent 的"工作合同"

定义目标和验收标准

progress.txt

实验笔记 — 已完成/进行中/阻塞

跨会话保持进度感知

AGENTS.md

规则手册 — 不可违反的约束

防止角色和行为漂移

CLAUDE.md

活计划 — 当前策略和下一步

支持上下文重置后恢复

CHANGELOG.md

可移植笔记 — 关键决策记录

支持团队交接和审计

状态外化 = 失忆安全。每次上下文重置后, Agent 从文件重建执行状态, 而非依赖记忆。

检查点与上下文重置

✗ 摘要式恢复

将长对话"摘要"后继续

问题:

- 摘要会丢失关键细节
- 目标在反复摘要中失真
- 对齐漂移通过摘要级联传播
- 无法用于多日任务

✓ 结构化重置

完全清空上下文 + 从外化文件重建

优势:

- 模拟"新同事接班"的交接模式
- 原始任务规范被完整重载
- 状态从结构化文件精确恢复
- 支持无限时长的任务执行

检查点频率的权衡: 太频繁→计算开销大; 太稀疏→故障时丢失进度多。实践中按"N 个有意义的状态变化"触发, 而非按时间间隔。

五大生产失败模式

上下文腐烂

模型性能在窗口填满前就开始退化

✓ 定期上下文重置 + 结构化交接

记忆漂移

Agent 将非典型交互中学到的捷径泛化 应用

✓ 记忆治理 + 定期审查学习内容

对齐漂移

原始目标在反复摘要中逐步失真

✓ 目标外化 + 每次推理重载原始任务

自验证失败

Agent 在完成 30% 时宣布"任务完成"

✓ 生成/评估分离, 禁止自我打分

成本爆炸

无预算/熔断器, 数小时内烧完数周 API 预算

✓ 硬性开支上限 + 自动熔断机制

怎么检测这些漂移? →

漂移检测方法

工具调用分布监控

追踪工具调用频率分布。正常时均匀，漂移时从"扁平"变"尖峰"(重复调用同一工具)

分布从均匀 → 集中 = 漂移信号

目标重锚检查

在每个检查点对比当前声明的目标与原始指令。自动标记偏差度

声明目标 vs 原始目标偏差 > 阈值 = 告警

推理密度阈值

监控输出中事实性推理 vs 模糊/重复内容的比例。下降意味着幻觉风险升高

密度跌破基线 = 标记幻觉风险升级

外部验证采样

随机抽取 Agent 的中间输出进行人工或 LLM-as-Judge 审查

准确率 < 阈值 = 触发干预

检测只是第一步, 完整的架构模式有哪些? →

五大生产架构模式

检查点恢复

每 N 个状态变化写入中间状态, 故障后从最近 检查点恢复

所有长时任务的基础

记忆分层治理

长期策展记忆 + 治理策略, 防止 Agent 学习非典型捷径

持久身份型 Agent

委托审批

人工暂停 Agent, 保持完整执行状态, 数小时后零计算成本恢复

高风险操作

环境流处理

事件流 Agent(内容审核/异常检测)+ 策略网关执行

7×24 监控场景

舰队编排

协调器分派任务给专家 Agent, 每个 Agent 有独立身份/策略/注册表

复杂多步 workflow

各大平台怎么支持这些能力? →

平台支持对比

平台	持久运行	状态管理	记忆系统	身份/安全
Google Agent Platform	天级自主运行	Agent Runtime 亚秒冷启动	Memory Bank 长期策展记忆	Agent Identity + Gateway
Anthropic Managed	30+ 小时验证	Brain/Hands/Session 三层分离	Session 事件日志	API 密钥分离 + 沙箱
Cursor Background	8h 重构 关盖不中断	云端检查点	本地/远端 无缝切换	用户级隔离

三个核心教训

1

"定义清晰的任务比做任务本身更难"

长时程 Agent 最大的挑战不是模型能力, 而是人类能否将工作规范化到足以让 Agent 独立运行一天的程度。这本质上是一个管理和流程设计问题。

2

外化一切, 信任架构而非上下文

目标、进度、规则、计划 — 所有关键信息都必须外化到持久存储。只存在于上下文窗口中的东西终将丢失。Event Log + 外化状态文件 + 检查点 = 失忆安全。

3

对齐是必要不充分条件

模型侧的 alignment 不能单独保证安全。生产级长时程 Agent 的稳健性几乎全部由 harness(外部基础设施)施加 — 护栏、审批门、预算上限、行为监控缺一不可。

总结

从"完成任务"到"持续经营"的路线图

